

Practical 7 OS

A computer system has four different types of resources (printer, ROM, hard disk and RAM) and it needs to complete execution of five processes (Google Drive, Firefox, Word Processor, Excel and PowerPoint)' The number of allocated resources, maximum needed resources to complete the execution and total available resources should be taken from the user. (Sample data for reference is given below). Calculate current need of resources by each process. Find whether these processes can be executed with available resources. If yes, show the correct order of process execution.

CODE:

```
tanuja@Tanuja: ~  
GNU nano 6.2  
#include <stdio.h>  
  
int main()  
{  
    int n = 5, m = 4; // 5 processes, 4 resources  
    int allocation[5][4], max[5][4], need[5][4];  
    int available[4];  
    int finish[5] = {0};  
    int safeSeq[5];  
    int i, j, k, count = 0;  
  
    printf("Enter Allocation Matrix:\n");  
    for(i=0; i<n; i++)  
    {  
        for(j=0; j<m; j++)  
        {  
            scanf("%d", &allocation[i][j]);  
        }  
    }  
  
    printf("Enter Maximum Matrix:\n");  
    for(i=0; i<n; i++)  
    {  
        for(j=0; j<m; j++)  
        {  
            scanf("%d", &max[i][j]);  
        }  
    }  
  
    printf("Enter Available Resources:\n");  
    for(i=0; i<m; i++)  
    {  
        scanf("%d", &available[i]);  
    }  
}
```

GNU nano 6.2

```
// Calculate Need Matrix
for(i=0;i<n;i++)
{
    for(j=0;j<m;j++)
    {
        need[i][j] = max[i][j] - allocation[i][j];
    }
}

printf("\nNeed Matrix:\n");
for(i=0;i<n;i++)
{
    for(j=0;j<m;j++)
    {
        printf("%d ", need[i][j]);
    }
    printf("\n");
}

while(count < n)
{
    int found = 0;

    for(i=0;i<n;i++)
    {
        if(finish[i] == 0)
        {
            int flag = 0;

            for(j=0;j<m;j++)
            {
                if(need[i][j] > available[j])
                {
                    flag = 1;
                    break;
                }
            }
        }
    }
}
```

tanuja@Tanuja: ~

GNU nano 6.2

bankers.c

```
        flag = 1;
        break;
    }
}

if(flag == 0)
{
    for(k=0;k<m;k++)
    {
        available[k] += allocation[i][k];
    }

    safeSeq[count++] = i;
    finish[i] = 1;
    found = 1;
}
}

if(found == 0)
{
    printf("System is NOT in safe state\n");
    return 0;
}

printf("\nSystem is in SAFE STATE\n");
printf("Safe Sequence: ");

for(i=0;i<n;i++)
{
    printf("P%d ", safeSeq[i]);
}

return 0;
}
```

OUTPUT:

```
Swap usage:    0%

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

This message is shown once a day. To disable it please create the
/home/tanuja/.hushlogin file.
tanuja@Tanuja:~$ nano bankers.c
tanuja@Tanuja:~$ gcc bankers.c
tanuja@Tanuja:~$ ./a.out
Enter Allocation Matrix:
0 1 0 2
2 0 0 1
3 0 2 1
2 1 1 0
0 0 2 0
Enter Maximum Matrix:
7 5 3 4
3 2 2 2
9 0 2 2
2 2 2 2
4 3 3 3
Enter Available Resources:
3 3 2 1

Need Matrix:
7 4 3 2
1 2 2 1
6 0 0 1
0 1 1 2
4 3 1 3

System is in SAFE STATE
Safe Sequence: P1 P3 P0 P2 P4 tanuja@Tanuja:~$ |
```